部署与复现指南

本文给出从裸板到可运行服务的完整步骤，以及验证结果的方法。全部脚本位于 `scripts/`，补丁位于 `patches/`。

1 环境要求

表 1 环境要求

项目	要求
硬件	MUSE Pi Pro（SpacemiT K1，8 核，8 GB 内存）
系统	Bianbu Linux（riscv64），内核与固件为出厂版本
编译工具链	出厂镜像自带 GCC 13.2；从源码编译需 `apt install gcc-14 g++-14 cmake`（板端源已含 14.2 与 cmake 3.28）
Python	3.10 及以上（仅量化脚本使用）
磁盘	至少 12 GB 可用（源码、f16 中间文件与量化产物）
网络	部署阶段不需要外网；模型与源码包可离线拷贝

2 交付物

表 2 交付物清单

文件	体积	用途
`../07_源码与二进制/llama-src-final-717a8ad8.tar.gz`	38,596,499 B	源码包，含 01—03 补丁（已合入），可重编
`../07_源码与二进制/llama-bin-chunk16-4436a9f4.tar.gz`	7,174,113 B	预编译二进制与库，可直接部署（deploy.sh 会自动解包）
`scripts/build.sh`	4,001 B	编译脚本，自动应用 07 号补丁
`scripts/quantize.sh`	1,835 B	量化脚本，从官方权重生成 Q4_0
`scripts/deploy.sh`	见 MANIFEST	部署脚本：解包交付二进制、推模型、**校验 SHA256（不符即停止）**、起服务
`scripts/verify.sh`	见 MANIFEST	验证脚本，运行基准并核对预期区间
`patches/01-07`	共 131 KB	7 个补丁：**01—03 已合入源码包**；04—06 是 K-quant 变体的重排补丁（扩展实验用，交付构建不含）；**07 由 `build.sh` 自动应用**

源码包与二进制的 SHA256 见 `MANIFEST.md`；交付模型与测试集的 SHA256 见 `../05_模型与测试集/05_模型与测试集.md`。

> **模型本体不随包**（体积原因）：百度网盘 `模型_网盘交付_2026-11-08.7z` `https://pan.baidu.com/s/1ya0FF80mw-qsCm6SydcUgA?pwd=qcnw` 提取码 `qcnw`；解压后放到 `../05_模型与测试集/models/` 下（`deploy.sh` 默认从那里取），或用 `MODELS_DIR=<目录>` 指定。详见 `05_模型与测试集.md` §2。

3 编译

```
tar xzf ../07_源码与二进制/llama-src-final-717a8ad8.tar.gz
cd spacemit-llama
bash ../scripts/build.sh . build
```

`build.sh` 依次完成：解包检查、补丁应用（若源码包未合入）、CMake 配置、编译、四项自检（Q4_1 repack、CONCAT、GQA、chunk16）。编译约 35 分钟，产物位于 `build/bin/`。

自检项与预期：

表 3 编译自检

检查项	预期
Q4_1 repack 符号	编译产物中存在对应 kernel
CONCAT 快速路径	源码 `ops.cpp` 含 fast path 分支
GQA 折叠	源码含折叠函数
chunk16	`GGML_CPU_CHUNK_MULT` 默认值为 16

4 量化

```
python convert_hf_to_gguf.py Qwen3.5-2B/ --outtype f16 --no-mtp --outfile 2b-f16.gguf
./build/bin/llama-quantize \
    --output-tensor-type Q6_K --token-embedding-type Q6_K \
    2b-f16.gguf Qwen3.5-2B-Q4_0-selfmade.gguf Q4_0 8
```

4B 模型将路径替换为 `Qwen3.5-4B/` 与对应文件名。量化参数必须保留 `--output-tensor-type Q6_K` 与 `--token-embedding-type Q6_K`，否则输出头精度下降。

产物校验值：

表 4 交付模型

模型	文件	字节数	SHA256
2B	`Qwen3.5-2B-Q4_0-selfmade.gguf`	1,202,487,136	`27a683603a6ae7165bb2e393b4f57bbdec24aaf5e11ee1c58c7d01ea23bb19df`
4B	`Qwen3.5-4B-Q4_0-selfmade.gguf`	2,543,899,168	`36a685821380dd641907fe35d7bbf70dff624d2d86f61ddb9104108d0931c09c`

5 部署

```
# 先下载模型到 ../05_模型与测试集/models/（见 05_模型与测试集.md 的链接）
# 首次使用先给脚本执行权限（压缩包在 Windows 上打包，解包后可能丢可执行位）
chmod +x scripts/*.sh 2>/dev/null || true
MODELS_DIR=../05_模型与测试集/models bash scripts/deploy.sh <板IP> Qwen3.5-2B-Q4_0-selfmade.gguf
```

`deploy.sh` 完成四步：解包交付二进制并推送、**校验 SHA256（本地或板端不符即停止，不会起服务）**、启动 `llama-server`、发送冒烟请求。单板部署约 5 分钟。

服务启动参数：

```
# deploy.sh 用的就是这一组（端口 8097，-fa 0 与报告/verify.sh 同口径）
llama-server -m <model> -c 4096 -t 7 --threads-batch 8 -ub 128 -b 2048 -fa 0 --port 8097 -np 1
```

> `-fa 0` **必须显式写**：该构建的 `-fa` 默认是 `auto`，而 auto 在本平台会解析成「开启」并走通用回退路径，长上下文会慢 15%—31%。`deploy.sh`、`verify.sh`、README 与报告都用 `-fa 0`，请保持一致。

长上下文场景将 `-c` 提升至 36864；并发场景将 `-np` 设为 4。参数含义与选型依据见技术报告 2.4 节。

6 验证

6.1 三分钟抽样（不想等全量时用这一条）

```
# 单点：4K 预填。2B 期望 ≈22.7 t/s（约 180 s），4B ≈9.0 t/s（约 450 s）
cd build/bin && LD_LIBRARY_PATH=. ./llama-bench -m <模型gguf> -p 4096 -n 0 -r 1 -t 8 -ub 128 -fa 0
```

档位	2B 期望	大致耗时
`-p 512`	25.2 t/s	20 s

档位	2B 期望	大致耗时
`-p 4096`（推荐抽样）	**22.7 t/s**	**3 分钟**
`-p 8192`	20.4 t/s	6.7 分钟
`-p 32768`	10.7 t/s	51 分钟

32K 那档不建议现场跑（要 51 分钟）——它的作用是"压力数据"，不是时延承诺。

> **口径提醒**：本表是 `-fa 0` 口径（与 `deploy.sh` / `verify.sh` 一致）。
> 技术报告 5.4 的长上下文表是 `-fa auto` 口径（2B 为 20.70/17.35/12.82/8.40），两者不可混读；
> 4K 差 10%（22.7 vs 20.70）、32K 差 28%（10.7 vs 8.40），差异全部来自 `-fa` 这个开关。
> 首 token 时延的绝对值可用 `05_模型与测试集/prompts/` 里的三份固定 prompt 复算（见该目录说明）。

6.2 完整验证

```
# 用法：bash scripts/verify.sh <模型gguf> [板端 llama.cpp 目录]
bash scripts/verify.sh Qwen3.5-2B-Q4_0-selfmade.gguf
```

`verify.sh` 运行两组基准：

表 5 验证基准与预期区间

基准	参数	2B 预期	4B 预期
预填	`-p 512 -n 128 -t 8 -ub 128 -r 3 -fa 0`	24.4—26.9 t/s	9.4—10.5 t/s
生成	`-p 0 -n 128 -t 7 -ub 128 -r 4 -fa 0`	4.2—4.5 t/s	1.9—2.1 t/s

首 token 验证（服务端）：

```
# 128 token 提示，冷启动（prompt 直接用包内固定文件，见 05_模型与测试集/prompts/ttft_128.txt）
python3 - <<'PY'
import json, urllib.request
p = open('../05_模型与测试集/prompts/ttft_128.txt', encoding='utf-8').read()
req = urllib.request.Request('http://127.0.0.1:8097/completion',
    data=json.dumps({'prompt': p, 'n_predict': 1, 'temperature': 0, 'cache_prompt': False}).encode(),
    headers={'Content-Type': 'application/json'})
t = json.load(urllib.request.urlopen(req, timeout=600))['timings']
print('prompt_n=%d  TTFT=%.2f s' % (t['prompt_n'], t['prompt_ms'] / 1000))
PY
```

2B 预期 **4.8—5.4 s**（本包实测 5.02 / 5.20 / 5.27 s），4B 预期 12.0—13.5 s。
口径与 `deploy.sh` 一致（`-t 7 --threads-batch 8 -ub 128 -b 2048 -fa 0`）。

7 证据复验

在提交包 `08_证据与复验/` 目录下执行：

```
cd 08_证据与复验
python tools/verify_claims.py    # 97 条 claim 逐条重算并校验证据 SHA256
```

预期输出 `97/97 通过`。登记表、证据文件与脚本均在本包内，不依赖外部仓库。

8 常见问题

编译失败。先确认已装 `gcc-14 g++-14 cmake`（出厂镜像只有 GCC 13.2、无 cmake）；编译须用 `gcc-14/g++-14`。若补丁应用失败，检查源码包是否为 `717a8ad8` 版本。

内存不足。4B 模型加载约需 2.8 GB 常驻内存；并发数超过 4 或上下文超过 16K 时需降低 `-c` 或 `-np`。

首 token 偏慢。确认 `-ub 128` 已设置；未设置时默认 512，首 token 时延约高出 10%—15%。

生成速度偏低。确认 `-t 7`（生成线程数）而非 8；8 线程在生成阶段会因缓存争用变慢。

模型文件名。赛题文本写作 `Qwen3.5-2B-Instruct`，官方模型包文件名为 `Qwen3.5-2B`（即指令微调版），本仓库沿用官方文件名。

9 目录结构

```
03_部署与复现/
├── 03_部署与复现.md
├── MANIFEST.md
├── scripts/
│   ├── build.sh
│   ├── quantize.sh
│   ├── deploy.sh
│   └── verify.sh
└── patches/
    ├── 01-q4_1-repack.patch
    ├── 02-concat-fastpath.patch
    ├── 03-gqa-fold.patch
    ├── 04-q5k-repack.patch
    ├── 05-q6k-repack.patch
    ├── 06-q3k-repack.patch
    └── 07-chunk16-chunkmult.patch
```

脚本与本文不一致时，以脚本内的实际参数为准；本文只描述预期行为与校验区间。